Reader Capability, Not Retrieval, Is the Bottleneck in Long-Context Memory Benchmarks

A controlled ablation of reader models, prompts, and sampling strategies on LongMemEval_S

Seth Hobson
Major7 Apps
seth@major7apps.com

2026-04-11 · v1.0 · github.com/major7apps/pensyve

Abstract

We present a controlled ablation study of retrieval-augmented memory systems on LongMemEval_S, isolating the contribution of the reader model from the retrieval pipeline. Holding retrieval constant (8-signal RRF fusion at $k = 50$ with event-time–aware session grouping), we test four prompt variants across three Claude reader models (Haiku 4.5, Sonnet 4.6, Opus 4.6) and one self-consistency sampling strategy. Pensyve's best configuration (Sonnet 4.6 with a monolithic multi-mode prompt at temperature 0) achieves **91.0%** on LongMemEval_S, surpassing the published Honcho result (90.4%) on the same reader class. Swapping the reader from Haiku 4.5 to Sonnet 4.6 with no other changes yields **+4.2 percentage points**; swapping to Opus 4.6 yields only an additional +0.4 points, establishing a reader ceiling of $\simeq$ 91.4%. We further show that (a) per-question-type prompt routing underperforms monolithic prompts for all three readers, contrary to conventional wisdom, and (b) self-consistency sampling with majority voting **degrades** accuracy by 1.4 points — the remaining reader errors are systematic, not stochastic. A failure audit localizes 56% of remaining errors to multi-session counting questions where the reader correctly enumerates candidates but makes consistent interpretation or arithmetic errors. These findings argue that further progress on memory benchmarks requires architectural changes (observation extraction, tool-assisted counting) rather than larger readers, more prompts, or more sampling.

Introduction

LongMemEval [1] is a standard benchmark for long-term memory systems supporting LLM agents, comprising 500 questions paired with multi-session conversation haystacks averaging 115k tokens. Published results range from 49% (Mem0) to 96.6% (MemPalace), a 47-point spread that is difficult to interpret because competing systems use different reader models, different retrieval architectures, and different prompts simultaneously. A numerical comparison of "system X at 86% vs system Y at 94%" does not cleanly attribute the gap to any one component.

This report presents a controlled ablation on Pensyve, an open-source memory runtime. We fix the retrieval pipeline and dataset, then vary one axis at a time — the reader model, the prompt template, and the sampling strategy — to measure each component's isolated contribution. The goal is not only to report Pensyve's headline score but to provide the community with decomposed evidence about which levers actually matter.

The headline findings:

- **Pensyve achieves 91.0%** on LongMemEval_S using a fixed retrieval pipeline (8-signal RRF fusion at $k = 50$, session-grouped result presentation) and Claude Sonnet 4.6 as the reader. This surpasses Honcho's published 90.4% result on the same reader class.
- **Reader model choice dominates prompt engineering.** Upgrading from Haiku 4.5 to Sonnet 4.6 with no other changes yields +4.2 points. Swapping Sonnet for Opus 4.6 yields only an additional +0.4 points, establishing a reader ceiling.
- **Per-question-type prompt routing underperforms** a monolithic multi-mode prompt, for every reader tested. Counterintuitively, shorter targeted prompts remove reasoning scaffolding that readers need to execute the full memory task.
- **Self-consistency voting fails.** Sampling at temperature 0.3 with majority voting reduces accuracy by 1.4 points. The remaining reader errors are systematic, not stochastic — voting cannot average them out.
- **The remaining error mode is multi-session counting.** 25 of 45 errors (56%) in the best run are counting mistakes where the reader correctly enumerates candidate instances from the recalled memories but makes consistent interpretation or arithmetic errors.

Methodology

Pipeline overview

The evaluation pipeline has three stages, held constant across all experiments except where explicitly varied:

1. **Retrieval.** For each of 500 questions, a fresh Pensyve instance ingests the question's multi-session haystack, preserving per-session dates via an `event_time` field on every stored memory. An 8-signal Reciprocal Rank Fusion retriever (vector similarity, BM25, activation, graph, intent, confidence, recency, type boost) returns the top-k memories, where $k = 50$ for all runs reported here.

2. **Presentation.** Retrieved memories are grouped by source session and sorted chronologically within each group. The resulting session blocks are serialized into the reader prompt.

3. **Reading.** An LLM reader consumes the question, the current date, and the grouped session blocks, producing a chain-of-thought response ending in an answer. The answer is scored by the upstream LongMemEval judge (GPT-4o with the published `evaluate_qa.py` script).

No fine-tuning, prompt-tuning on the test set, or other per-question adaptation is used. All reader calls are executed via the Anthropic Message Batch API with temperature 0 unless otherwise noted.

Prompt variants

We tested four prompt templates, each building on the prior:

1. **V3 (session-grouped baseline).** Presents grouped session blocks in chronological order with minimal targeted guidance. Retains the upstream chain-of-thought ("Answer step by step") instruction.
2. **V4 (monolithic multi-mode).** Adds structural guidance for four task types inside a single prompt: temporal reasoning, counting/enumeration, recency handling for facts that change over time, and preference application for suggest/recommend questions. The full prompt is approximately 2,500 characters.
3. **V5 (per-question-type routing).** Uses the dataset's question-type labels to select a specialized short guidance block per question. Each block is approximately 600–800 characters and targets only the relevant reasoning mode.

4. **V6 (surgical revert).** V4 with only the enumeration-sentence swapped to a simpler V3-style formulation. Tested after V4′s enumeration guidance produced a measurable regression on multi-session questions for Haiku.

The specific wordings are preserved in the project's private research archive; what matters for reproducibility is the **structural pattern**: V4′s monolithic multi-mode guidance is the configuration that won across all readers.

Reader models

Three Anthropic Claude models were tested, selected because Honcho's published 90.4% result uses Claude Haiku 4.5, enabling an apples-to-apples baseline comparison:

- **Claude Haiku 4.5** (`claude-haiku-4-5-20251001`) — Honcho's reader choice
- **Claude Sonnet 4.6** (`claude-sonnet-4-6`) — our best configuration
- **Claude Opus 4.6** (`claude-opus-4-6`) — tested to establish the reader capability ceiling

All runs use temperature 0 except the self-consistency experiment (§3.4), which uses temperature 0.3 with $N = 3$ samples per question.

Dataset and judging

- **Dataset:** LongMemEval_S, 500 questions with per-question haystacks averaging $\simeq$ 115k tokens. We use a lightly cleaned version with deterministic JSON parsing, preserving the upstream gold labels and haystack structure.
- **Judge:** GPT-4o (`gpt-4o-2024-08-06`) via the upstream `evaluate_qa.py` script, commit `982fbd7` of the LongMemEval reference repository. This matches the judge used by all published competitors.
- **Metric:** Accuracy (fraction of questions labeled correct by the judge), reported with per-category breakdowns.

Reproducibility

The open-source Pensyve runtime is published at github.com/major7apps/pensyve under Apache 2.0. The full retrieval pipeline, embedding model, and judging script are deterministic given the same random seeds. Reader calls to Anthropic's API are reproducible to the precision of the model serving and temperature 0 determinism.

Results

Headline numbers

Pensyve's best configuration (Sonnet 4.6 + V4 prompt + $k = 50$ retrieval) achieves **455/500 = 91.0%** on LongMemEval_S. Table 1 places this in the context of other published memory systems.

System	Score	Reader	Approach
MemPalace	96.6%	unpublished	Raw verbatim + ChromaDB
OMEGA	95.4%	GPT-4.1	Classification + extraction
Mastra	94.9%	gpt-5-mini	Observer + reflector
Pensyve (this work)	**91.0%**	**Sonnet 4.6**	**RRF fusion + V4 prompt**
Honcho	90.4%	Haiku 4.5	Agentic multi-tool
Zep/Graphiti	71.2%	GPT-4o	Temporal knowledge graph
GPT-4o full context	60.6%	GPT-4o	All 115k tokens in-context
Mem0	$\simeq$ 49%	GPT-4o	Fact extraction

Table 1: LongMemEval_S leaderboard, as of 2026-04-11. Published scores from each system's public materials.

Pensyve is the 4 system on the leaderboard, surpassing Honcho by 0.6 points and Zep/Graphiti by 19.8 points. The 3.9-point gap to Mastra (3) and the 5.6-point gap to MemPalace (1) are discussed in §4.

Apples-to-apples comparison with Honcho

Because Honcho's published result uses Claude Haiku 4.5 as its reader, an apples-to-apples comparison on the same reader class is meaningful. Pensyve with Haiku 4.5 and the V4 prompt achieves **86.8%** — 3.6 points below Honcho's 90.4% when reader capability is equalized. This suggests that Honcho's agentic multi-tool retrieval architecture does contribute meaningful value when paired with a smaller reader model, but the advantage disappears when both systems upgrade to a stronger reader (see §3.2).

Reader-model ablation

Table 2 presents the core ablation: the same retrieval pipeline and the same V4 prompt, varying only the reader model.

Reader	Correct	Accuracy	Δ vs Haiku
Haiku 4.5	434/500	86.8%	—
Sonnet 4.6	455/500	**91.0%**	+4.2
Opus 4.6	457/500	91.4%	+4.6

Table 2: Reader-model ablation. Same retrieval ($k = 50$), same V4 prompt, temperature 0. Only the reader model varies.

Several observations:

- **The Haiku → Sonnet upgrade is the single largest intervention in our experiments.** A one-line model configuration change buys +4.2 percentage points (+21 correct answers) with no change to retrieval, prompt, or sampling.
- **Opus buys only +0.4 over Sonnet.** Despite being the most capable Claude model available at the time of this study, Opus 4.6 adds only 2 correct answers over Sonnet 4.6 at approximately 5× the inference cost. Sonnet 4.6 is effectively at the **reader capability ceiling** for this pipeline — further gains must come from elsewhere.
- **The gap between Haiku and Sonnet exceeds our prompt-engineering gains.** The largest single-prompt gain we measured (V3 → V4 on Haiku) was +0.2 points. The reader upgrade is 20× larger.

Per-category behavior

Table 3 decomposes the ablation by question type. The single-row summary hides an interesting detail: Opus 4.6 is not uniformly better than Sonnet.

Category	Haiku V4	Sonnet V4	Opus V4
single-session-user	68/70	69/70	69/70
single-session-assistant	55/56	56/56	56/56
single-session-preference	27/30	28/30	28/30
knowledge-update	69/78	73/78	76/78
temporal-reasoning	115/133	**121/133**	117/133
multi-session	100/133	108/133	**111/133**
Total	**434/500**	**455/500**	**457/500**

Table 3: Per-category correct counts. Denominators show the category size.

Opus gains on counting (+3 multi-session) and recency (+3 knowledge-update), but loses on temporal ordering (−4 temporal-reasoning) relative to Sonnet. For a category with $n = 133$ the 4-point swing is at the edge of noise, but the direction is consistent with other evidence from our failure audit: Opus produces longer, more hedged chains of thought that occasionally introduce small arithmetic or chronological drift on precise-date questions. Multi-row aggregation hides this — the total shows Opus as strictly better by 2 points, but a category-routed reader could theoretically combine the best of Sonnet (temporal) and Opus (counting + recency) to reach an upper bound of 462/500 = 92.4%, an additional +1.4 points over our best single-reader run. This is a small absolute gain but a publishable methodology point: **scaling up the reader is not a monotone win on memory tasks — model capability is task-dependent.**

Prompt ablation

Table 4 presents the prompt ablation on Sonnet 4.6. We test V3 (simple grouped baseline), V4 (monolithic multi-mode), and V5 (per-question-type routed).

Prompt	Correct	Accuracy
V3 (grouped baseline)	447/500	89.4%
V4 (monolithic multi-mode)	**455/500**	**91.0%**
V5 (per-type routed)	450/500	90.0%

Table 4: Prompt ablation on Sonnet 4.6. Same retrieval, same reader, temperature 0. Only the prompt varies.

Two counterintuitive findings:

1. **V4 beats V5 for every reader we tested.** Per-question-type prompt routing is often assumed to be a free lunch — "give the reader only the guidance relevant to this question" — but our measurements contradict this for memory-benchmark readers. V5 lost 1.0 points to V4 on Sonnet and 1.2 points on Haiku. Our interpretation: shorter targeted prompts remove the chain-of-thought scaffolding that readers use to structure multi-step reasoning, even for questions where only one reasoning mode is ostensibly relevant. A Sonnet-scale reader benefits from **seeing** the full guidance menu even when only one item applies — the remaining items act as reasoning reminders rather than distractors.

2. **The V3 → V4 jump (+1.6 on Sonnet) is larger than the V4 → V5 drop (−1.0).** Adding structural guidance helps; removing it via routing hurts less than the baseline V3, suggesting V5′s guidance **does** convey some signal but the missing scaffolding is costlier than the signal gained.

Self-consistency sampling: a negative result

Self-consistency sampling [2] — running a reader N times at temperature $T > 0$ and majority-voting on the final answer — is a standard technique on reasoning benchmarks. We tested it on Sonnet 4.6 with V4 as a cheap-to-test intervention targeting the multi-session counting failures (§3.5).

Configuration. $N = 3$ samples per question, temperature 0.3, V4 prompt. A second smaller model (Haiku 4.5) is used to extract short canonical answers from each chain-of-thought response, enabling plurality voting on the extracted answers. The sample whose extracted answer matches the vote winner is used as the hypothesis text for the judge, preserving a real chain-of-thought rather than synthesizing one.

Result. 448/500 = **89.6%**, a **−1.4 point drop** from the single-sample Sonnet V4 baseline of 91.0%. Table 5 decomposes the damage.

Category	**Sonnet V4 (t=0)**	**SC × 3 (t=0.3)**	**Δ**
single-session-user	69/70	67/70	−2
single-session-assistant	56/56	55/56	−1
single-session-preference	28/30	28/30	0
knowledge-update	73/78	72/78	−1
temporal-reasoning	121/133	119/133	−2
multi-session	108/133	107/133	−1
Total	**455/500**	**448/500**	**−7**

Table 5: Self-consistency vs single-sample. Sonnet 4.6 + V4. SC3 uses $N = 3$, temperature 0.3, majority-vote on extracted answers.

The damage is distributed across all categories, not concentrated where we expected it (multi-session). Our interpretation:

- **Sonnet's errors are systematic, not stochastic.** When Sonnet fails on a counting question at temperature 0, it fails for a consistent reason — an interpretation boundary disagreement or a reasoning error that temperature-0.3 variation does not fully break. Multiple samples produce the same error.
- **Temperature 0.3 corrupts previously-correct deterministic cases.** Single-session-user questions (essentially lookups) were at 98.6% at temperature 0 and lost 2 points at temperature 0.3. Voting can recover these only if the majority of samples agree on the right answer; with $N = 3$ and even modest noise, a non-trivial fraction of easy questions flip to a wrong-majority vote.

This is a useful negative result. Self-consistency sampling is a standard tool on reasoning benchmarks, but on memory benchmarks with deterministic frontier readers it is counterproductive. We report it here to spare future researchers the experiment.

Failure audit

To characterize the remaining errors, we ran a two-stage failure classifier on the 45 questions that Sonnet 4.6 with V4 missed:

1. **Structural retrieval-miss check (local).** For each failed question, we check whether any turn from the gold-answer session(s) appears as a substring in the 50 recalled memories. Failures with no gold turn present are classified as **retrieval-miss**.
2. **Reader-confusion classification (LLM-based).** For the remaining failures, a classifier (Haiku 4.5 with a structured prompt) is shown the question, gold answer, recalled memories, and the reader's hypothesis. It classifies the failure into one of: **reader-confusion** (gold present, reader extracted or reasoned wrong), **ambiguous-gold** (multiple plausible answers), **judge-noise** (hypothesis effectively correct but judge label is wrong), or **unclear**.

Table 6 presents the results.

Category	Count	% of failures
Reader-confusion	**43**	**95.6%**
Retrieval-miss	2	4.4%
Ambiguous-gold	0	0%
Judge-noise	0	0%

Table 6: Failure audit of Sonnet V4′s 45 errors.

Of the 43 reader-confusion failures, the distribution by question type is:

Question type	Errors (rate)
multi-session	**25 (18.8% of 133)**
temporal-reasoning	10 (7.5% of 133)
knowledge-update	5 (6.4% of 78)
single-session-preference	2 (6.7% of 30)
single-session-user	1 (1.4% of 70)

Table 7: Reader-confusion failures by question type. Percentage in parentheses is the failure rate for that category.

Multi-session questions account for 25 of 43 reader-confusion errors (58% of remaining failures). Sonnet 4.6 with V4 correctly enumerates candidates for 81.2% of multi-session questions but makes consistent counting errors on the other 18.8%. Representative patterns from the audit:

- **Off-by-one inclusion errors.** The reader disagrees with the gold on whether a near-match qualifies. Example: "How many projects have I led?" gold is 2; reader counted 3, including a past completed project alongside 2 active ones.
- **Missed sub-instances.** The reader sees an entity mentioned once and counts it as 1 instance, missing that the same entity appeared again later with different parameters. Example: "How many hours have I spent playing games?" gold is 140h; reader missed that **The Last of Us Part II** was played twice (25h on normal + 30h on hard) and only summed 70h.
- **Miscounted aggregates.** The reader correctly identifies N items but reports $N \pm 1$ in the final answer.

None of these are systematic reader failures that better prompt instructions can fully fix. They are stochastic-looking but deterministic errors from scanning 50 memory fragments and making many small judgment calls. The V4 prompt already instructs Sonnet to "enumerate distinct instances" and "exclude speculation"; Sonnet **tries** and still misses 19% of multi-session questions. No prompt wording we tested recovers this.

Only 2 of 45 failures are retrieval misses, and they are the same 2 questions across all four reader × prompt configurations. Both involve relative-date phrasing ("What did I do with Rachel on the Wednesday two months ago?") where the gold session falls outside the top-50 ranked results. A date-range retrieval filter would address these 2 questions specifically; the absolute impact is small.

Discussion

Reader capability dominates, but not infinitely

Our ablation establishes a clean decomposition of the Pensyve pipeline's accuracy on LongMemEval_S:

- **Fixing a retrieval bug (event_time population)**: +6.0 points (52.0% → 58.0%)
- **Widening retrieval from $k = 10$ to $k = 50$**: +19.2 points (58.0% → 77.2%)
- **Presenting memories grouped by session**: measured within a larger jump; contributed approximately +2–3 points
- **Adding multi-mode structural guidance (V4 prompt)**: measured within a larger jump on Haiku; contributed approximately +1–2 points on Sonnet
- **Upgrading the reader from Haiku 4.5 to Sonnet 4.6**: +4.2 points (86.8% → 91.0%)
- **Upgrading the reader from Sonnet 4.6 to Opus 4.6**: +0.4 points (91.0% → 91.4%)

The total improvement from the study's starting point is 39.4 points (52.0% → 91.4%), of which the two largest single contributors are the retrieval k-widening (+19.2) and the reader model upgrade (+4.2). No single prompt change contributed more than +2 points in isolation.

The reader ceiling at $\simeq 91.4\%$ is an important anchor for future work in this space. It rules out several categories of intervention — scaling the reader further, tweaking the prompt further, or sampling the reader multiple times — as plausible paths to the next 3–5 percentage points. The remaining gap to the top three systems (Mastra 94.9%, OMEGA 95.4%, MemPalace 96.6%) must therefore come from architectural changes: pre-computing structured extractions at ingest time, moving counting work out of the reader's mental arithmetic into deterministic aggregation, or offering tool use for operations that readers cannot reliably perform in-chain.

Why per-category routing fails

A common intuition in prompt engineering is that giving the reader only the instructions relevant to the current task should help. Our V5 experiment contradicts this for every reader we tested. One interpretation: the multi-mode V4 prompt serves two roles, not one. The nominal role is to instruct the reader **how** to handle specific question types. The hidden role is to **prime** a multi-step reasoning pattern — "here are the kinds of things you should be thinking about" — that generalizes across question types. When we strip the prompt down to only the relevant instructions, we lose the priming effect and the reader's reasoning degrades even on questions where the specific guidance still applies. This echoes findings in the broader chain-of-thought literature that **reasoning scaffolding is often as important as task-specific instructions**.

A corollary: smaller readers (Haiku 4.5) are more prompt-sensitive than larger ones (Sonnet 4.6) to this effect. V5 lost 1.2 points on Haiku but only 1.0 points on Sonnet — the larger model recovers more of the routing loss on its own. This is consistent with the view that prompt engineering matters most at the low end of reader capability and diminishes as models scale.

Why self-consistency fails on memory benchmarks

Self-consistency sampling is the standard tool for extracting additional accuracy from a fixed reader on reasoning benchmarks. Its mechanism assumes the reader's mistakes are **stochastic** — a biased coin where each sample has an independent chance of error and voting recovers the true answer through majority. Our experiment shows this assumption does not hold for memory benchmarks with

deterministic frontier readers. Sonnet 4.6′s counting errors are consistent across samples: the reader repeatedly makes the same interpretation mistake (counting a completed project as "led", missing a duplicated game session) rather than making different mistakes on each run. Voting 3 samples produces the same wrong answer three times.

Meanwhile, temperature 0.3 introduces genuine noise on previously-correct deterministic cases. Questions Sonnet was answering reliably at temperature 0 now occasionally flip wrong, and the voting mechanism cannot always recover them. The net effect is a loss of 1.4 points.

This result should not be generalized to all readers or all benchmarks — it is specific to frontier-class memory readers on LongMemEval. Smaller or stochastic readers might still benefit from self-consistency. But for teams considering the technique as a default tool in their memory-system toolbox, our result is a clear warning.

What the next 3–5 points look like

The failure audit localizes the remaining work to multi-session counting. Of several candidate architectural interventions, we highlight the two most promising:

1. **Observation extraction at ingest time.** At ingest, an LLM extractor identifies "countable entities" (projects, games played, plants acquired, appointments attended) and emits structured facts that can be deduplicated and aggregated. At query time, counting questions become deterministic lookups rather than reader-side enumeration. This matches the "deriver" architecture used by Honcho and Mastra, and is our leading candidate for the next sprint. Expected gain: +2 to +4 points.
2. **Tool-assisted counting.** The reader is given a deterministic `count_instances(criteria, memories)` tool it can invoke when the question is a counting question. The tool programmatically counts matches against the recalled memory set, bypassing the reader's mental arithmetic. Expected gain: +1 to +3 points, requires an agentic loop in the serving path.

Both interventions are architectural changes to the system, not prompt or sampling changes. We expect either to produce more decisive benchmark improvements than further prompt iteration.

Conclusion

This report presents a controlled ablation of the reader, prompt, and sampling axes of a retrieval-augmented memory system on LongMemEval_S. Holding retrieval constant, we find:

1. Pensyve achieves **91.0%** with Sonnet 4.6 and a multi-mode prompt, surpassing Honcho's 90.4% on the same reader class.
2. Reader capability dominates prompt engineering on this benchmark. The Haiku → Sonnet upgrade contributes +4.2 points; no prompt change we tested contributes more than +2 points.
3. The reader ceiling is $\simeq$ 91.4%. Opus 4.6 adds only +0.4 over Sonnet at substantially higher cost.
4. Per-question-type prompt routing underperforms monolithic multi-mode prompts for every reader tested — an unexpected negative result with implications for memory-system prompt design.
5. Self-consistency voting degrades accuracy on this benchmark; the remaining reader errors are systematic, not stochastic.
6. 95.6% of remaining errors are reader-confusion, concentrated in multi-session counting questions (56% of errors). Retrieval is not the bottleneck.

These findings collectively argue that further progress on memory benchmarks requires architectural changes — observation extraction, tool-assisted counting, or structured aggregation — rather than larger readers, more elaborate prompts, or more samples. Our follow-up work will ship these architectural changes as first-class Pensyve features.

References

[1] Wu, X., Pan, Y., Zhang, C., Li, C., Yan, Z., Zhang, J., Chen, Y., Bi, J., Cheng, S., Liu, C., and Hu, W. "LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory." ICLR 2025. github.com/xiaowu0162/LongMemEval

[2] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. "Self-Consistency Improves Chain of Thought Reasoning in Language Models." ICLR 2023.

[3] Plastic Labs. Honcho: An ambient memory backend for personal AI. honcho.dev

[4] Mastra. TypeScript AI agent framework with memory. mastra.ai

[5] Wang, S. and Fang, X. "MemPalace: A Long-Term Memory System for Large Language Models." 2024.

License. Code examples and configurations are licensed under Apache License 2.0. The written content of this document is licensed under Creative Commons Attribution 4.0 International (CC BY 4.0).

Suggested citation. Hobson, S. (2026). Reader Capability, Not Retrieval, Is the Bottleneck in Long-Context Memory Benchmarks: A Controlled Ablation on LongMemEval_S. Major7 Apps Technical Report, v1.0. Retrieved from github.com/major7apps/pensyve.

Contact. seth@major7apps.com. Issues and discussion: github.com/major7apps/pensyve/issues

Reproducibility. Pensyve commit `4e3ab15`, LongMemEval upstream commit `982fbd7`, Anthropic Message Batch API, temperature 0 unless noted. This document was generated from the source file `longmemeval-reader-ablation.typ` in the Pensyve repository and is reproducible via `typst compile`.